Effective Computation of a Feedback Arc Set Using Machine Learning-Enhanced PageRank Batching

Georgios Papadakis, Dr. Panagiotis Lionakis, Dr. Michalis Mountantonakis

Computer Science Department, University of Crete, Greece

October 2026

Abstract

Computing a minimum Feedback Arc Set (FAS) is a foundational NP-hard combinatorial optimization problem with pervasive applications in hierarchical graph drawing, causal inference, and network cycle elimination [15, 23, 12]. The recently proposed PageRankFAS algorithm achieves state-of-the-art reductions in FAS cardinality—often eliminating over 50% more redundant arcs than classical heuristics such as GreedyFAS and SortFAS—by mapping each Strongly Connected Component (SCC) to a directed line digraph $L(G)$ and scoring edges through undamped PageRank traversals [12]. Despite its superior decycling quality, the baseline algorithm follows a strict single-edge deletion policy: after removing only one maximal-scoring edge, the line digraph and its stationary distributions must be completely recomputed [12]. Because constructing and traversing $L(G)$ scales with $O(m + \sum_{u \in V}[d^+(u) \times d^-(u)])$, this continuous recalculation incurs a severe super-linear overhead, causing prohibitive runtimes on dense or large-scale networks [12, 22].

To overcome this scalability barrier, this thesis introduces **ML-Batching-PageRankFAS**, an intelligent batch-decycling framework that evaluates and eliminates multiple feedback arcs simultaneously within each SCC [25]. By extracting a candidate set of top-K PageRank-ranked edges per component, our framework models structural competition as a supervised pairwise classification task [25]. We extract a seven-dimensional feature vector for every candidate pair—capturing directed shortest-path reachability, shared boundary endpoints, common in/out-neighborhood topologies, and PageRank score differentials—and deploy a cost-sensitive Random Forest classifier trained on empirical decycling simulations [7, 25]. Predicted conflict probabilities are mapped into an auxiliary undirected conflict graph G_c, where a Maximum Independent Set (MIS) is computed to isolate a safe, mutually non-interfering batch of edges for concurrent removal [25]. This batching paradigm substantially diminishes the number of global line-graph reconstructions and PageRank iterations, preserving the optimal decycling efficacy of PageRankFAS while achieving significant runtime speedups across synthetic benchmarks, tournament graphs, and real-world web networks [12, 25].

1 Introduction

1.1 Context and Motivation

In graph theory and network analysis, a directed graph $G = (V, E)$ consists of a set of vertices V and a set of directed edges (arcs) E, with $|V| = n$ and $|E| = m$. A directed cycle is a non-empty sequence of directed edges that starts and ends at the same vertex with no other repeated vertices [12]. A directed graph that contains no directed cycles is termed a Directed Acyclic Graph (DAG) [12]. DAGs possess a natural topological ordering, making them essential mathematical abstractions for modeling causal structures, dependency graphs, schedule scheduling, and hierarchical architectures [23, 10].

When an arbitrary directed graph contains cycles, transforming it into a DAG requires the identification and resolution of cyclical feedback dependencies [12]. Formally, a *Feedback Arc Set* (FAS) is a subset of edges $F \subseteq E$ whose removal renders the remaining subgraph $G' = (V, E \setminus F)$ acyclic [12]. The *Minimum Feedback Arc Set problem* seeks to find a subset F of minimum cardinality $|F|$ [15]. Computing a minimum FAS is fundamentally dual to the *Maximum Acyclic Subgraph* (MAS) problem, which seeks the largest edge-induced acyclic subgraph $E_{MAS} = E \setminus F$ [3], as well as the *Optimal Linear Arrangement* (LA) problem, which orders the vertices such that the number of backward-pointing

arcs is minimized [11, 22]. As proven by Karp in his seminal 1972 work, computing the minimum FAS is NP-hard [15]. Furthermore, the problem is APX-hard, meaning that finding a polynomial-time approximation scheme with arbitrary precision is computationally intractable unless P = NP.

The practical necessity of computing high-quality, minimal feedback arc sets spans numerous scientific and technological domains:

- **Hierarchical Graph Visualization:** The classical Sugiyama framework [23] and the modern Ortali-Tollis framework [18] for drawing directed graphs hierarchically require cycle removal as their indispensable first phase [10, 17, 12]. In visualization workflows, edges in F are not discarded but inverted in direction to produce a *Feedback Set* (FS) [10, 12]. Because every minimal-cardinality FAS is guaranteed to be a valid FS, minimizing $|F|$ directly prevents visual artifacts and minimizes upward edge crossings in layered layouts [10, 12].

- **Complex Network Dynamics and Social Systems:** In social network analysis, cycle breaking is used to model hierarchy discovery, trust propagation, and influence blocking [13, 22]. It is equally pivotal in limiting the viral diffusion of misinformation across social platforms, where breaking cyclic echo chambers prevents runaway cascading effects [9, 22].

- **Data Mining and Web Search:** On massive web graphs, calculating linear rankings and applying layered label propagation algorithms relies heavily on removing backwards-referencing link cycles to infer natural page importance and authoritative hierarchies [4, 22].

- **Chemical and Process Engineering:** Minimum FAS calculations are utilized in the steady-state simulation of chemical plants and flowsheets, where tearing cyclic streams is mandatory for numerical convergence [2].

1.2 Existing Algorithms and the PageRank Paradigm

Due to the NP-hardness of the problem, exact algorithms are computationally prohibitive for large graphs. Recently, Baharev et al. proposed an exact algorithm utilizing Integer Linear Programming (ILP) that lazily enumerates simple cycles [2]. While capable of proving optimal FAS solutions for small-to-medium sparse graphs ($n \leq 120$), its runtime scales exponentially, frequently exceeding dozens of hours on dense benchmarks or timing out completely on larger instances [2, 12].

Consequently, practical applications rely on polynomial-time heuristics [12]. Simple techniques, such as Depth-First Search decycling (*dfsFAS*), identify and strip back-edges in $O(n+m)$ time [24], but yield highly suboptimal FAS sizes that depend heavily on arbitrary search tree traversals [12]. The landmark *GreedyFAS* heuristic introduced by Eades, Lin, and Smyth [11] constructs a linear arrangement by iteratively peeling sink vertices, source vertices, and vertices maximizing the degree differential $\delta(u) = d^+(u) - d^-(u)$ in linear time $O(n+m)$ [11, 22]. Brandenburg and Hanauer extended the tournament-based KwikSortFAS algorithm [1] to general digraphs via *SortFAS*, an insertion-sort heuristic operating in $O(n^2)$ time [6, 22]. In an extensive empirical study on web-scale graphs, Simpson et al. established that GreedyFAS and SortFAS provided the strongest balance between speed and FAS reduction among classical methods [22].

To push solution quality beyond these traditional bounds, Geladaris, Lionakis, and Tollis introduced **PageRankFAS** [12]. The foundational intuition of PageRankFAS is that an edge's structural contribution to directed cycles can be evaluated through stationary random-walk distributions [8, 19, 12]. Because feedback arcs exist strictly within the Strongly Connected Components (SCCs) of a graph, the algorithm decomposes G into its maximal SCCs in $O(n+m)$ time via Tarjan's algorithm [24, 12]. Within each SCC s_i, it constructs the directed line graph (or line digraph) $L(s_i) = (V_L, E_L)$ [21, 12]. In $L(s_i)$, every vertex represents a directed edge of s_i, and an edge is directed from node (u,v) to (v,w) if and only if those edges form a directed path of length two in the original graph [12]. A directed cycle in s_i corresponds to an identical cycle in $L(s_i)$ [12].

By executing an undamped ($\alpha = 1.0$) PageRank calculation on $L(s_i)$, probability mass naturally circulates through cyclic topologies [19, 12]. Edges embedded in a higher volume of interlocking cycles accumulate greater PageRank mass [12]. Selecting the line graph vertex with the highest score identifies the edge whose deletion breaks the maximum concentration of cycles [12]. Empirical evaluations in [12] revealed that PageRankFAS produces dramatic improvements over prior art: on standard graph drawing benchmarks, it reduces the final FAS cardinality by up to 55% compared to GreedyFAS and SortFAS, and successfully uncovered smaller FAS sizes on SNAP web graphs than previously recorded in the literature [22, 12].

1.3 The Recomputation Bottleneck

Despite its exceptional solution quality, PageRankFAS exhibits a critical computational bottleneck: its *single-edge removal paradigm* [12]. In the baseline framework, after five PageRank iterations are evaluated over $L(s_i)$, the algorithm extracts only the single edge with the highest score, appends it to the FAS, and removes it from G [12]. However, deleting even a single edge alters the cycle space, shifts in- and out-degree distributions, and can fragment the underlying SCC into multiple smaller components [12]. Consequently, the line digraph and its PageRank vector become obsolete immediately after each removal [12].

To maintain optimality, the baseline algorithm must repeat the entire pipeline—SCC decomposition, line digraph synthesis, and five iterations of PageRank—for *every individual edge removed* [12]. The construction and traversal of a line digraph $L(s_i)$ requires:

$$O\left(m_{s_i} + \sum_{u \in V_{s_i}} \left[d^+(u) \times d^-(u)\right]\right) \tag{1}$$

time and space complexity [12]. If a graph requires thousands of edge eliminations to become acyclic, executing thousands of full line-graph regenerations imposes a prohibitive super-linear runtime overhead [12]. As observed by Geladaris et al., while PageRankFAS completes in less than one second on graphs with up to 1,000 nodes, its execution time escalates rapidly on larger, denser networks, leading to timeouts on web-scale datasets where GreedyFAS terminates in seconds [22, 12].

1.4 The Machine Learning Batching Paradigm

This thesis addresses the fundamental question: *Can we safely remove multiple high-scoring feedback arcs in a single iteration without degrading the near-optimal FAS size of PageRankFAS?*

Naively removing an arbitrary batch of top-scoring edges (e.g., the top-K PageRank edges) leads to severe algorithmic degradation [25]. Two high-scoring edges may participate in the same minimal cycle; deleting both constitutes a redundant removal, unnecessarily inflating the FAS size. Conversely, if two candidate edges reside in disjoint cycles or distinct topological neighborhoods, their concurrent removal is mutually safe and independent [25].

To exploit this structural decoupling, we propose **ML-Batching-PageRankFAS**, a machine learning-augmented framework that replaces blind greedy batching with predictive topological conflict resolution [25]. Instead of removing only the top-scoring edge from each SCC, we extract a batch of the top-K candidate edges ranked by their PageRank scores [25]. To prevent redundant removals, we formulate candidate interaction as a supervised pairwise classification problem: given two candidate edges e_A and e_B, does their concurrent removal produce a suboptimal FAS size compared to sequential recalculated removal? [25]

We capture the spatial and cyclic interdependencies of candidate pairs through seven localized topological features:

1. Direct node intersection (identifying shared incident endpoints) [25].
2. Directed shortest-path distance from e_A to e_B [25].

3. Directed shortest-path distance from e_B to e_A (evaluating cycle closure between the pair) [25].
4. Common successor count (shared forward neighborhoods) [25].
5. Common predecessor count (shared backward neighborhoods) [25].
6. Absolute PageRank score differential $|PR(e_A) - PR(e_B)|$ [25].
7. PageRank score ratio $\min(PR)/\max(PR)$ [25].

Using these features, a cost-sensitive Random Forest classifier predicts the conflict probability P_{conflict} for every candidate pair in the top-K set [7, 25]. These pairwise probabilities populate an auxiliary undirected *conflict graph* $G_c = (V_c, E_c)$, where vertices represent candidate edges and edges denote predicted structural conflicts parameterized by a confidence threshold τ [25]. By computing the Maximum Independent Set (MIS) of G_c, we isolate the largest mutually safe, conflict-free subset of edges [25]. Removing this entire safe batch simultaneously cuts down the number of line-graph reconstructions by an order of magnitude while maintaining solution quality competitive with the baseline algorithm [25].

1.5 Thesis Contributions

The core contributions presented in this thesis are summarized as follows:

- **Formulation of Batch-Decycling in Line Digraphs:** We establish a formal batching framework for line-digraph-based Feedback Arc Set computation, transitioning PageRankFAS from a sequential single-edge removal paradigm to an iterative multi-edge batching architecture [12, 25].
- **Pairwise Structural Feature Engineering:** We design and implement a compact, computationally efficient feature extraction pipeline that models topological conflict and cycle overlap between candidate edge pairs without requiring global cycle enumeration [25].
- **Cost-Sensitive Machine Learning Formulation:** We generate a labeled dataset of structural conflict scenarios on synthetic digraphs and train a balanced Random Forest model that prioritizes the minimization of false negatives, protecting the decycling process from erroneous cycle-breaking predictions [25].
- **Graph-Theoretic Conflict Resolution via MIS:** We map the machine learning conflict probabilities into an auxiliary conflict graph and apply Maximum Independent Set approximation to mathematically guarantee that concurrent removals are pairwise non-conflicting [25].
- **Comprehensive Empirical Benchmarking:** We perform extensive experimental evaluations on synthetic Erdős-Rényi graphs, Baharev tournament benchmarks, and SNAP web graphs [2, 16, 12]. We demonstrate that ML-Batching-PageRankFAS drastically accelerates decycling speed while closely matching the minimal FAS cardinality of the baseline algorithm across diverse graph topologies [25].

1.6 Thesis Organization

The remainder of this thesis is structured as follows. Section 2 provides a detailed examination of the existing PageRankFAS algorithm, formulating its mathematical foundations, line digraph transformation, undamped PageRank convergence, and algorithmic complexity limits. Section 3 details the architecture of our proposed ML-Batching-PageRankFAS framework, explaining the top-K extraction process, feature extraction formulas, dataset synthesis, Random Forest training, and MIS conflict resolution. Section 4 presents experimental benchmarks across synthetic, tournament, and web-scale graphs, analyzing the trade-offs between execution runtime and FAS cardinality across varying conflict thresholds τ. Finally, Section 5 concludes the thesis and highlights avenues for future research in scalable graph decycling.

2 Existing Algorithm (PageRankFAS)

2.1 Problem Formulation and Mathematical Foundations

Let $G = (V, E)$ be a directed graph (digraph) where V represents the set of vertices with $|V| = n$, and $E \subseteq V \times V$ represents the set of directed edges (arcs) with $|E| = m$ [12]. A directed path in G is a sequence of vertices $(v_1, v_2, \ldots, v_k)$ such that $(v_i, v_{i+1}) \in E$ for all $1 \leq i < k$ [12]. A directed cycle is a directed path that begins and ends at the same vertex, such that $v_1 = v_k$ and the vertices $v_1, \ldots, v_{k-1}$ are distinct [12]. A directed graph containing no directed cycles is a Directed Acyclic Graph (DAG) [12].

A *Feedback Arc Set* (FAS) is a subset of directed edges $F \subseteq E$ whose removal from G eliminates all directed cycles, yielding an acyclic subgraph $G' = (V, E \setminus F)$ [12]. The *Minimum Feedback Arc Set problem* seeks to find a feedback arc set F^* of minimum cardinality:

$$F^* = \arg\min_{F \subseteq E} \{|F| \mid G \setminus F \text{ is a DAG}\} \tag{2}$$

The minimum FAS problem is fundamentally equivalent to two classical graph optimization problems:

- **Maximum Acyclic Subgraph (MAS):** Finding an edge-induced acyclic subgraph $G' = (V, E')$ such that $|E'|$ is maximized [3]. The optimal solutions satisfy $|E_{MAS}| = |E| - |F^*|$ [3, 12].
- **Optimal Linear Arrangement (LA):** Finding a bijective topological layout $\pi : V \to \{1, 2, \ldots, n\}$ that minimizes the number of backward-pointing arcs, defined as $F_\pi = \{(u, v) \in E \mid \pi(u) > \pi(v)\}$ [11, 22]. The set of backward arcs F_π strictly forms a valid FAS [11, 12].

In hierarchical graph drawing algorithms, such as the Sugiyama framework [23] and the Ortali-Tollis framework [18], edges identified in a feedback arc set are not permanently removed; instead, their orientations are reversed [10, 17, 12]. A set of arcs whose reversal renders the graph acyclic is called a *Feedback Set* (FS) [10, 12]. While an arbitrary FAS is not guaranteed to be an FS (for instance, reversing every edge of an isolated cycle may create a reversed cycle), every *minimal cardinality* FAS is guaranteed to be a valid FS [10, 12]. Consequently, computing a minimum FAS is mathematically equivalent to computing a minimum FS [10, 12].

The decision version of the minimum FAS problem was proven NP-complete by Karp in 1972 [15, 14]. Exact solutions can be computed via Integer Linear Programming (ILP) with lazy cycle generation, as demonstrated by Baharev et al. [2]. However, because the number of simple cycles in a digraph can grow exponentially with graph density, exact ILP approaches require excessive execution times (often thousands of seconds on graphs with only 100 vertices) and fail to scale to medium or large networks [2, 12].

To approximate the minimum FAS in polynomial time, several heuristics have been developed:

- **Depth-First Search (dfsFAS):** Executes a DFS traversal in $O(n+m)$ time and removes all identified back-edges [24]. Although rapid, the solution quality depends on the arbitrary starting vertex and traversal order, producing suboptimal FAS sizes [12].
- **GreedyFAS:** Introduced by Eades, Lin, and Smyth [11], this algorithm constructs a linear arrangement in $O(n+m)$ time by repeatedly peeling sinks, sources, and the vertex maximizing the degree differential $\delta(u) = d^+(u) - d^-(u)$ [11, 22].
- **SortFAS:** Introduced by Brandenburg and Hanauer [6] as an extension of the tournament 3-approximation algorithm KwikSortFAS [1], SortFAS iteratively inserts vertices into a linear arrangement to minimize backward-pointing arcs in $O(n^2)$ time [6, 22].

While Simpson, Srinivasan, and Thomo established that GreedyFAS and SortFAS offer practical runtimes on web-scale graphs [22], their greedy local criteria frequently remove substantially more arcs than necessary [12].

2.2 Strongly Connected Component (SCC) Decomposition

A directed graph G is strongly connected if there exists a directed path between every ordered pair of vertices $u, v \in V$ [24, 12]. The Strongly Connected Components (SCCs) of G are its maximal strongly connected subgraphs [24, 12]. If each SCC is contracted into a single super-vertex, the resulting condensation graph G^{SCC} is acyclic by definition [12].

Because edges that span between distinct SCCs can never participate in any directed cycle, every directed cycle in G is strictly confined within a single SCC [12]. Therefore:

$$F^*(G) = \bigcup_{s_i \in \mathrm{SCC}(G)} F^*(s_i) \tag{3}$$

Decomposing G into its components $(s_1, s_2, \ldots, s_c)$ using Tarjan's linear-time algorithm ($O(n+m)$) [24] isolates the cycle space. Any component with $|V_{s_i}| \leq 1$ or $|E_{s_i}| = 0$ is acyclic and can be pruned immediately [12]. This modular decomposition restricts subsequent line digraph synthesis and PageRank evaluations strictly to cyclic subgraphs, avoiding wasteful global computations [12].

2.3 Directed Line Graph Transformation

The cornerstone of the PageRankFAS framework is evaluating edge centrality by transforming the input graph into a directed line graph, also known as a *line digraph* $L(G) = (V_L, E_L)$ [21, 12]. In this dual representation:

- Each directed edge $e = (u, v) \in E$ in the original graph becomes a distinct vertex in the line digraph:
$$V_L = E \tag{4}$$
- A directed edge exists from vertex $e_1 = (u, v)$ to vertex $e_2 = (x, y)$ in $L(G)$ if and only if the head of e_1 coincides with the tail of e_2 in G (i.e., $v = x$):
$$E_L = \{((u, v), (v, w)) \mid (u, v) \in E \text{ and } (v, w) \in E\} \tag{5}$$

Consequently, every directed edge in $L(G)$ corresponds to a directed path of length two in G [12]. Furthermore, any directed cycle of length k in G, given by $(v_1, v_2, \ldots, v_k, v_1)$, maps to a directed cycle of identical length in $L(G)$:

$$((v_1, v_2), (v_2, v_3), \ldots, (v_k, v_1), (v_1, v_2)) \tag{6}$$

The structural dimensions of $L(G)$ depend on the vertex degree distributions of G [21, 12]. While the number of vertices in $L(G)$ is exactly m, each vertex $v \in V$ in the primal graph contributes $d^-(v) \times d^+(v)$ directed paths of length two [12]. Therefore, the number of edges in $L(G)$ is:

$$|E_L| = \sum_{v \in V} \left[d^-(v) \times d^+(v) \right] \tag{7}$$

Constructing $L(G)$ requires iterating over all vertices $v \in V$ and forming the Cartesian product between their incoming and outgoing edges, yielding a spatial and temporal complexity of:

$$O\left(m + \sum_{v \in V} \left[d^-(v) \times d^+(v) \right] \right) \tag{8}$$

The formal algorithmic procedure for line digraph construction is detailed in Algorithm 1.

Listing 1: Directed Line Graph Construction

Algorithm 1: BUILDLINEDIGRAPH(G)
Input: Directed graph $G = (V, E)$
Output: Directed line graph $L(G) = (V_L, E_L)$

$V_L \leftarrow E$
$E_L \leftarrow \emptyset$
for each $v \in V$ **do**
    for each $(u,v) \in E$ **do**
        for each $(v,w) \in E$ **do**
            $E_L \leftarrow E_L \cup \{((u,v),(v,w))\}$
return $L(G) = (V_L, E_L)$

2.4 Undamped PageRank on the Line Digraph

PageRank was originally formulated by Brin and Page [8, 19] to determine the structural importance of web pages by modeling the stationary distribution of a random surfer over a hyperlink graph. In PageRankFAS, PageRank is applied to the line digraph $L(s_i)$ of an SCC s_i to quantify the cycle participation of edges in the original graph [12]. Because random walks along $L(s_i)$ navigate directed paths of length two, the probability mass naturally circulates through directed cycles [12]. Edges that participate in a high concentration of overlapping cycles receive recurrent transitions from adjacent cyclic edges, accumulating higher stationary probability scores [12].

Standard web PageRank formulations employ a damping factor $\alpha \approx 0.85$ to guarantee ergodicity and prevent random walks from becoming trapped in sink vertices or disconnected clusters [8, 19]. However, in PageRankFAS, PageRank is strictly executed within the line digraph of a Strongly Connected Component [12]. Because every vertex in an SCC has at least one incoming and one outgoing edge ($d^+(v) \geq 1, d^-(v) \geq 1$), every node in $L(s_i)$ has outgoing arcs, and probability mass cannot escape the cyclic component [12]. Introducing a damping factor $\alpha < 1.0$ would inject an artificial uniform probability restart across all edges, distorting the pure topological cycle centrality [12]. Consequently, PageRankFAS sets $\alpha = 1.0$, completely eliminating teleportation [12].

The PageRank vector $\mathbf{p} \in \mathbb{R}^{|V_L|}$ is initialized uniformly:

$$p_0(e) = \frac{1}{|V_L|}, \quad \forall e \in V_L \tag{9}$$

During each power-iteration step $t \in \{1, \ldots, k\}$, each vertex $e \in V_L$ distributes its accumulated score equally among its successors:

$$p_t(e') = \sum_{e \in \text{Pred}(e')} \frac{p_{t-1}(e)}{d_L^+(e)} \tag{10}$$

If a vertex $e \in V_L$ acts as a sink ($d_L^+(e) = 0$), its probability mass is distributed uniformly across all vertices in V_L to preserve total mass conservation ($\sum_{e \in V_L} p_t(e) = 1.0$) [12].

While standard PageRank on arbitrary networks may require dozens of power iterations to reach strict numerical convergence, Geladaris et al. demonstrated empirically that edge centrality rankings stabilize rapidly on line digraphs [12]. Exactly five power iterations ($k = 5$) are sufficient to produce stationary scores that reliably identify the edge maximizing cycle reduction [12]. The formal procedure is presented in Algorithm 2.

Listing 2: Undamped PageRank on Line Digraph ($\alpha = 1.0$)

Algorithm 2: PAGERANKLINEDIGRAPH($L(G), k$)
Input: Line digraph $L(G) = (V_L, E_L)$, iteration bound $k = 5$
Output: Stationary score vector $\mathbf{p} \in \mathbb{R}^{|V_L|}$

$N \leftarrow |V_L|$
for each $e \in V_L$ **do**
    $p_0(e) \leftarrow \frac{1}{N}$
for $t \leftarrow 1$ **to** k **do**
    for each $e \in V_L$ **do**

```
        p_t(e) ← 0
    for each e ∈ V_L do
        d_L^+(e) ← |{e' ∈ V_L | (e,e') ∈ E_L}|
        if d_L^+(e) > 0 then
            for each e' ∈ Succ(e) do
                p_t(e') ← p_t(e') + p_{t-1}(e) / d_L^+(e)
        else
            for each e' ∈ V_L do
                p_t(e') ← p_t(e') + p_{t-1}(e) / N
return p_k
```

2.5 Iterative Edge Removal Procedure

The baseline PageRankFAS framework executes an iterative decycling procedure [12]. In the preprocessing phase, the algorithm scans G for self-loops (edges of the form $(u,u) \in E$). Because self-loops form trivial cycles of length one and must be broken by any valid FAS, they are stripped immediately, appended to F, and removed from G [12].

Next, the algorithm enters the primary decycling loop, which repeats until G contains no remaining cycles:

1. Decompose G into its set of Strongly Connected Components $\mathscr{S} = \{s_1, s_2, \ldots, s_c\}$ [24, 12].
2. For each non-trivial component s_i containing at least two vertices:
   (a) Construct the component line digraph $L(s_i) = \textsc{BuildLineDigraph}(s_i)$ [12].
   (b) Compute the edge centrality vector $\mathbf{p} = \textsc{PageRankLineDigraph}(L(s_i), 5)$ [12].
   (c) Identify the edge e^* corresponding to the vertex with the maximum PageRank score [12]:

$$e^* = \arg \max_{e \in V(L(s_i))} \mathbf{p}(e) \tag{11}$$

   (d) Append e^* to the feedback arc set ($F \leftarrow F \cup \{e^*\}$) and delete e^* from G [12].
3. Re-evaluate whether G is a DAG; if cycles persist, repeat the process [12].

The complete baseline algorithm is formalized in Algorithm 3.

Listing 3: Baseline PageRankFAS Decycling Procedure

Algorithm 3: $\textsc{PageRankFAS}(G,k)$
Input: Directed graph $G = (V,E)$, PageRank iterations $k = 5$
Output: Feedback Arc Set $F \subseteq E$

```
F ← ∅
E_self ← {(u,u) ∈ E}
if E_self ≠ ∅ then
    F ← F ∪ E_self
    G ← G \ E_self
while G is not a Directed Acyclic Graph do
    S ← StronglyConnectedComponents(G)
    for each s_i = (V_{s_i}, E_{s_i}) ∈ S do
        if |V_{s_i}| ≤ 1 then
            continue
        L(s_i) ← BuildLineDigraph(s_i)
        if |V(L(s_i))| = 0 then
            continue
        p ← PageRankLineDigraph(L(s_i), k)
```

15: $e^* \leftarrow \arg\max_{e \in V(L(s_i))} \mathbf{p}(e)$
16: $F \leftarrow F \cup \{e^*\}$
17: $G \leftarrow G \setminus \{e^*\}$
18: **return** F

2.6 Computational Complexity and Scalability Bottlenecks

The time complexity of a single iteration of PageRankFAS within an SCC $s_i = (V_{s_i}, E_{s_i})$ is governed by the line digraph construction and the subsequent power iterations [12]:

$$T_{\text{iter}}(s_i) = O\left(|E_{s_i}| + \sum_{v \in V_{s_i}} \left[d^-(v) \times d^+(v)\right]\right) \tag{12}$$

Because $k = 5$ is a small constant, the PageRank computation runs in linear time with respect to the size of $L(s_i)$ [12]. However, the edge cardinality $|E(L(s_i))| = \sum[d^- \times d^+]$ introduces super-linear scaling when nodes possess high in- and out-degrees [12]. In the worst-case scenario of dense subgraphs, $|E(L(s_i))|$ scales quadratically up to $O(|E_{s_i}|^2)$.

The fundamental scalability bottleneck arises from the *sequential single-edge removal paradigm* [12]. Let $c = |F|$ denote the total number of feedback arcs that must be removed to render G acyclic. Because the removal of a single arc e^* mutates paths of length two, alters local cycle frequencies, and potentially fragments an SCC into multiple smaller components, the baseline framework invalidates the previous stationary distributions [12]. Consequently, the algorithm recomputes SCC decompositions, reconstructs line digraphs, and re-executes five iterations of PageRank after *every single edge removal* [12].

The cumulative worst-case time complexity of PageRankFAS across the full decycling process scales to:

$$T_{\text{total}}(G) = O\left(c \cdot \left(m + \sum_{v \in V} \left[d^-(v) \times d^+(v)\right]\right)\right) \tag{13}$$

On sparse graphs with fewer than 1,000 vertices, this overhead is negligible, completing in under one second [12]. However, on large or dense networks where c spans thousands of edges, this continuous recomputation becomes computationally prohibitive [12]. In empirical benchmarks on web-scale graphs, such as those evaluated by Simpson et al. [22], PageRankFAS suffers from extreme execution runtimes and timeouts [12].

While PageRankFAS achieves exceptional FAS minimality—frequently producing feedback arc sets that are 50% smaller than those produced by GreedyFAS or SortFAS on graph drawing benchmarks—its sequential recalculation policy limits its practical deployment [12]. This limitation underscores the need for our proposed methodology: an intelligent batching mechanism capable of predicting mutually safe subsets of candidate edges to eliminate concurrently in each iteration, amortizing the super-linear cost of line digraph construction without compromising FAS solution quality [25].

3 Our Approach (ML-Batching-PageRankFAS)

3.1 Overview and Batching Formulation

While the baseline PageRankFAS algorithm achieves state-of-the-art reductions in Feedback Arc Set cardinality [12], its practical utility is constrained by its sequential single-edge removal paradigm [12] In each decycling round, the algorithm reconstructs the directed line digraph $L(s_i)$ and evaluates five undamped PageRank iterations, only to eliminate a single maximal-scoring edge before discarding all calculated stationary distributions [12] Because the line digraph edge cardinality scales with the node degree product sum $\sum_{u \in V}[d^+(u) \times d^-(u)]$, re-executing this pipeline thousands of times on large or dense networks incurs an intractable computational overhead [12, 22]

The core motivation of **ML-Batching-PageRankFAS** is to amortize the costly line-digraph synthesis and PageRank power iterations by safely eliminating a batch of multiple feedback arcs per iteration [25] However, executing naive batching—such as greedily removing all top-K PageRank-ranked edges within a component—leads to severe algorithmic degradation [25]

- **Cycle Co-membership and Edge Redundancy:** If two candidate edges e_A and e_B lie on the same simple directed cycle, eliminating the higher-scoring edge e_A instantly breaks that cycle [12]
- **Topological Decoupling and Component Fragmentation:** Conversely, if two candidate edges reside in mutually disjoint cycles or distant topological neighborhoods within the same Strongly Connected Component (SCC), their cycle-breaking actions are structurally orthogonal [25]

Evaluating all 2^K possible edge subsets within a candidate pool is computationally infeasible [25] To circumvent this combinatorial explosion, our approach models candidate competition as a *pairwise structural conflict prediction* problem [25] By extracting a localized topological feature vector for every pair of candidate edges, a cost-sensitive Random Forest classifier predicts the posterior probability that their concurrent deletion produces a suboptimal FAS [7, 25] These probabilities are mapped to an auxiliary undirected *conflict graph* G_c, where a Maximum Independent Set (MIS) isolates a guaranteed mutually non-conflicting "safe batch" of edges for concurrent removal [15, 5, 25]

3.2 SCC Decomposition and Candidate Edge Extraction

In alignment with the structural properties established in Section 2, directed cycles exist exclusively within the Strongly Connected Components of a graph [24, 12] Our algorithm begins by partitioning $G = (V,E)$ into its maximal SCCs $\mathscr{S} = \{s_1, s_2, \ldots, s_c\}$ using Tarjan's linear-time algorithm [24] Components containing fewer than two vertices cannot contain cycles and are immediately bypassed [12, 25]

For each non-trivial SCC $s_i = (V_{s_i}, E_{s_i})$, we construct its directed line digraph $L(s_i) = (V_L, E_L)$ where $V_L = E_{s_i}$ [21, 12] We then execute five power iterations of undamped PageRank ($\alpha = 1.0$) across $L(s_i)$ [19, 12]

$$p_t(e') = \sum_{e \in \text{Pred}(e')} \frac{p_{t-1}(e)}{d_L^+(e)}, \quad t \in \{1, \ldots, 5\} \tag{14}$$

Instead of extracting only the single vertex maximizing the stationary score vector $\mathbf{p}$, we extract the top-K candidate edges possessing the highest PageRank values [25]

$$C(s_i) = \left(e_{(1)}, e_{(2)}, \ldots, e_{(K)}\right), \quad \text{where } p(e_{(1)}) \geq p(e_{(2)}) \geq \cdots \geq p(e_{(K)}) \tag{15}$$

The parameter K represents the candidate batch window per SCC (with $K = 20$ configured as the default operational batch ceiling) [25] If an SCC contains only a single candidate ($|C(s_i)| = 1$), the algorithm skips feature extraction and immediately removes that edge, gracefully falling back to baseline behavior [25]

3.3 Pairwise Structural Feature Engineering

For every unique pair of candidate edges (e_1, e_2) within the candidate pool $C(s_i)$, we formulate a predictive representation of their structural interrelationship [25] To establish strict input consistency for the machine learning model, we designate the edge with the higher PageRank score as e_A and the secondary edge as e_B, such that $PR(e_A) \geq PR(e_B)$ [25] Let $e_A = (u, v)$ and $e_B = (x, y)$, where u, x denote the source endpoints (tails) and v, y denote the target endpoints (heads) in G [25]

We extract a specialized, seven-dimensional topological feature vector $\blacksquare(e_A, e_B) \in \mathbb{R}^7$ designed to capture cycle co-membership, endpoint proximity, shared neighborhood density, and relative centrality magnitude without requiring global cycle enumeration [25]

1. **Shared Endpoint Count (ϕ_1):** Quantifies the direct vertex intersection between the two directed arcs:
$$\phi_1(e_A, e_B) = |\{u, v\} \cap \{x, y\}| \in \{0, 1, 2\} \tag{16}$$
A value of $\phi_1 \geq 1$ signifies that e_A and e_B are incident upon a common vertex (sharing a tail, a head, or forming an adjacent chain) [25]

2. **Forward Geodesic Distance (ϕ_2):** Measures the length of the shortest directed path in G from the head of e_A to the tail of e_B:
$$\phi_2(e_A, e_B) = \begin{cases} \text{dist}_G(v, x) & \text{if a directed path from } v \text{ to } x \text{ exists in } G \\ |V| & \text{otherwise (unreachable)} \end{cases} \tag{17}$$
Setting unreachable distances to the total graph order $|V| = n$ imposes a finite upper penalty [25]

3. **Backward Geodesic Distance (ϕ_3):** Measures the length of the shortest directed path in G from the head of e_B to the tail of e_A:
$$\phi_3(e_A, e_B) = \begin{cases} \text{dist}_G(y, u) & \text{if a directed path from } y \text{ to } u \text{ exists in } G \\ |V| & \text{otherwise (unreachable)} \end{cases} \tag{18}$$
Topological Implication of ϕ_2 and ϕ_3: If both $\phi_2 < |V|$ and $\phi_3 < |V|$, there exists a directed path from v to x and a directed path from y to u [25]
$$\ell_{\text{cycle}} = \text{dist}_G(v, x) + \text{dist}_G(y, u) + 2 \tag{19}$$
When both geodesic distances are short, the edges are bound to tight, localized cycles, indicating that eliminating e_A will immediately resolve the cycle containing e_B [12, 25]

4. **Common Successor Count (ϕ_4):** Measures the cardinality of shared forward out-neighbors from the target nodes:
$$\phi_4(e_A, e_B) = |\text{Succ}(v) \cap \text{Succ}(y)| \tag{20}$$
where $\text{Succ}(w) = \{z \in V \mid (w, z) \in E\}$ [25]

5. **Common Predecessor Count (ϕ_5):** Measures the cardinality of shared backward in-neighbors into the source nodes:
$$\phi_5(e_A, e_B) = |\text{Pred}(u) \cap \text{Pred}(x)| \tag{21}$$
where $\text{Pred}(w) = \{z \in V \mid (z, w) \in E\}$ [25]

6. **PageRank Score Differential (ϕ_6):** The absolute magnitude difference between their stationary PageRank centralities:
$$\phi_6(e_A, e_B) = |PR(e_A) - PR(e_B)| = PR(e_A) - PR(e_B) \geq 0 \tag{22}$$
A small differential indicates that both edges participate in an equivalent density of cycles, whereas a large differential implies that e_A is topologically dominant [12, 25]

7. **PageRank Score Ratio (ϕ_7):** The normalized ratio of PageRank centralities:
$$\phi_7(e_A, e_B) = \begin{cases} \frac{\min(PR(e_A), PR(e_B))}{\max(PR(e_A), PR(e_B))} = \frac{PR(e_B)}{PR(e_A)} & \text{if } PR(e_A) > 0 \\ 0.0 & \text{otherwise} \end{cases} \tag{23}$$
Because $\phi_7 \in [0, 1]$, this feature provides scale-invariant centrality normalization across components of varying size [25]

3.4 Training Data Generation and Cost-Sensitive Model Training

3.4.1 Empirical Ground-Truth Dataset Generation

Supervised learning requires ground-truth labels indicating whether concurrent edge deletions preserve downstream FAS optimality [25] To generate this dataset, we synthesize a corpus of $M = 3,000$ random directed graphs $G \sim G(n,p)$ using the Erdős-Rényi model, sampling graph sizes $n \in [30,100]$ and edge connection probabilities $p \in [0.05,0.15]$ [25] Trivial self-loops are stripped, and acyclic graphs are discarded [25]

For each valid training graph G:

1. We execute the exact baseline $\text{PAGERANKFAS}(G)$ algorithm to compute the baseline minimum FAS cardinality [12, 25]
$$S_{\text{base}} = |\text{PAGERANKFAS}(G)| \tag{24}$$
2. We isolate the top-K candidate edges ($K = 10$) across all components based on their line-digraph PageRank scores [25]
3. For each unique pair $(e_A, e_B) \in \binom{C}{2}$, we extract the feature vector $\blacksquare(e_A, e_B) \in \mathbb{R}^7$ [25]
4. We simulate concurrent removal by instantiating a modified test graph $G_{\text{test}} = (V, E \setminus \{e_A, e_B\})$ and decycling it to completion using the baseline algorithm [25]
$$S_{\text{test}} = 2 + |\text{PAGERANKFAS}(G_{\text{test}})| \tag{25}$$
5. The ground-truth conflict label $y \in \{0,1\}$ is assigned based on the optimality condition [25]
$$y(e_A, e_B) = \begin{cases} 1 \quad \text{(Conflict)} & \text{if } S_{\text{test}} > S_{\text{base}} \\ 0 \quad \text{(Safe)} & \text{if } S_{\text{test}} \leq S_{\text{base}} \end{cases} \tag{26}$$

Empirical generation over 3,000 synthetic networks yields a naturally imbalanced dataset comprising approximately 72.2% Safe pairs ($y = 0$) and 27.8% Conflict pairs ($y = 1$) [25]

3.4.2 Cost-Sensitive Random Forest Architecture

We deploy a Random Forest classifier [7] parameterized with $T = 100$ decision tree estimators [20, 25] The feature matrix is partitioned into an 80% training set and a 20% testing set using stratified sampling to maintain the $72.2\%/27.8\%$ class distribution across both partitions [20, 25]

In graph decycling, classification errors exhibit severe *asymmetric costs* [25]

- **False Positive (Type I Error):** The model predicts an edge pair is conflicting ($P_{\text{conflict}} \geq \tau$) when it is safe. The framework will separate the edges into distinct batches, resulting in a minor loss in batching speed but leaving final FAS quality intact [25]
- **False Negative (Type II Error):** The model predicts an edge pair is safe ($P_{\text{conflict}} < \tau$) when it is conflicting. Both edges are deleted concurrently, irrevocably inflating the final FAS size with redundant arcs [25]

To heavily suppress dangerous False Negatives, the Random Forest is trained using cost-sensitive balanced class weighting [20, 25] Each training sample of class $c \in \{0,1\}$ is assigned an optimization weight $w_c = \frac{N_{\text{total}}}{2 \cdot N_c}$, heavily penalizing misclassifications of the minority Conflict class during node splitting [20, 25] Rather than outputting hard binary predictions, the model outputs the calibrated posterior conflict probability:
$$P_{\text{conflict}}(e_A, e_B) = \frac{1}{T} \sum_{t=1}^{T} P_t(y = 1 \mid \blacksquare) \tag{27}$$

This exposes a tunable decision threshold $\tau \in [0.0, 1.0]$: a pair is designated as conflicting if and only if $P_{\text{conflict}} \geq \tau$ [25] Lower thresholds (e.g., $\tau = 0.30$) force highly risk-averse batching that strictly protects FAS cardinality, whereas higher thresholds (e.g., $\tau = 0.70$) permit aggressive batching to maximize runtime acceleration on massive graphs [25]

3.5 Predictive Conflict Graph and Maximum Independent Set (MIS)

Once the Random Forest model generates pairwise conflict probabilities for all $\binom{|C(s_i)|}{2}$ combinations within candidate set $C(s_i)$, the algorithm must select a maximal subset of candidate edges that can be deleted simultaneously without mutual interference [25]

We map these interactions into an undirected *conflict graph* $G_c = (V_c, E_c)$ defined as follows [25]

- The vertex set V_c corresponds bijectively to the extracted top-K candidate edges:

$$V_c = C(s_i) \tag{28}$$

- An undirected edge connects two candidate vertices e_A and e_B in G_c if and only if their predicted conflict probability meets or exceeds the operational threshold τ:

$$E_c = \{\{e_A, e_B\} \subseteq V_c \mid P_{\text{conflict}}(e_A, e_B) \geq \tau\} \tag{29}$$

An *independent set* in G_c is a vertex subset $I \subseteq V_c$ such that no two vertices in I are joined by an edge in E_c [15]. By construction, every pair of edges in I satisfies $P_{\text{conflict}} < \tau$, mathematically guaranteeing that no predicted pairwise structural conflicts exist within I [25] To maximize the number of edges removed in the current iteration, we solve the *Maximum Independent Set* (MIS) problem on G_c [15, 5, 25]

$$I^* = \arg \max_{\substack{I \subseteq V_c \\ \forall u,v \in I, \{u,v\} \notin E_c}} |I| \tag{30}$$

Although finding the MIS of an arbitrary graph is NP-hard [15], the order of our conflict graph is strictly bounded by $|V_c| = K \leq 20$ [25] On graphs of this modest size, greedy minimum-degree approximation algorithms [5] compute the maximum independent set in microseconds, introducing zero observable computational overhead [25]

3.5.1 Fallback Safety Mechanism

In dense cyclic components where edges exhibit severe cyclic coupling, the conflict graph G_c may degenerate into a near-clique, potentially yielding an empty independent set under aggressive pruning [25] To guarantee continuous algorithmic progress and prevent infinite loops, the framework incorporates an automatic fallback:

$$I_{\text{safe}} = \begin{cases} I^* & \text{if } I^* \neq \emptyset \\ \{e_{(1)}\} & \text{if } I^* = \emptyset \end{cases} \tag{31}$$

where $e_{(1)}$ is the single edge possessing the highest PageRank score in $C(s_i)$ [25] This fallback ensures that the algorithm matches or exceeds the per-iteration progress of baseline PageRankFAS under all topological conditions [12, 25] Every edge in I_{safe} is appended to the FAS and removed simultaneously from G [25]

3.6 Algorithmic Formulation and Complexity Analysis

The end-to-end decycling framework combines candidate extraction, pairwise topological feature computation, machine learning conflict prediction, and MIS batch extraction into an integrated loop [25] The feature extraction and pairwise scoring procedure is formalized in Algorithm 4, and the main decycling engine is detailed in Algorithm 5.

Listing 4: Pairwise Conflict Feature Extraction and Prediction

Algorithm 4: PREDICTCONFLICTPROBABILITY$(G, e_1, e_2, PR, \mathscr{M}_{\text{RF}})$
Input: Directed graph $G = (V, E)$, candidate edges e_1, e_2, PageRank vector PR, RF model $\mathscr{M}_{\text{RF}}$
Output: Posterior conflict probability $P_{\text{conflict}} \in [0, 1]$

if $PR(e_1) \geq PR(e_2)$ **then**
    $e_A \leftarrow e_1$; $e_B \leftarrow e_2$
else
    $e_A \leftarrow e_2$; $e_B \leftarrow e_1$
$(u, v) \leftarrow e_A$; $(x, y) \leftarrow e_B$; $N \leftarrow |V|$
$\phi_1 \leftarrow |\{u, v\} \cap \{x, y\}|$
$\phi_2 \leftarrow$ (directed path $v \rightsquigarrow x$ in G) ? $\text{dist}_G(v, x) : N$
$\phi_3 \leftarrow$ (directed path $y \rightsquigarrow u$ in G) ? $\text{dist}_G(y, u) : N$
$\phi_4 \leftarrow |\text{Succ}(v) \cap \text{Succ}(y)|$
$\phi_5 \leftarrow |\text{Pred}(u) \cap \text{Pred}(x)|$
$\phi_6 \leftarrow PR(e_A) - PR(e_B)$
$\phi_7 \leftarrow (PR(e_A) > 0)$? $(PR(e_B)/PR(e_A)) : 0.0$
$\blacksquare \leftarrow (\phi_1, \phi_2, \phi_3, \phi_4, \phi_5, \phi_6, \phi_7)$
$P_{\text{conflict}} \leftarrow \mathscr{M}_{\text{RF}}$.PREDICTPROBABILITY($\blacksquare$)
return P_{conflict}

Listing 5: ML-Batching-PageRankFAS Algorithm

Algorithm 5: ML-BATCHING-PAGERANKFAS(G, K, τ)
Input: Directed graph $G = (V, E)$, batch bound $K = 20$, conflict threshold $\tau \in [0, 1]$
Output: Feedback Arc Set $F \subseteq E$

$F \leftarrow \emptyset$
$\mathscr{M}_{\text{RF}} \leftarrow$ LOADPRETRAINEDRANDOMFOREST()
$E_{\text{self}} \leftarrow \{(u, u) \in E\}$
if $E_{\text{self}} \neq \emptyset$ **then**
    $F \leftarrow F \cup E_{\text{self}}$; $G \leftarrow G \setminus E_{\text{self}}$
while G is not a Directed Acyclic Graph **do**
    $\mathscr{S} \leftarrow$ STRONGLYCONNECTEDCOMPONENTS(G)
    for each $s_i = (V_{s_i}, E_{s_i}) \in \mathscr{S}$ **do**
        if $|V_{s_i}| \leq 1$ **then continue**
        $L(s_i) \leftarrow$ BUILDLINEDIGRAPH(s_i)
        if $|V(L(s_i))| = 0$ **then continue**
        $\mathbf{p} \leftarrow$ PAGERANKLINEDIGRAPH($L(s_i), 5$)
        $C \leftarrow$ EXTRACTTOPKCANDIDATES($\mathbf{p}, K$)
        if $|C| = 1$ **then**
            $F \leftarrow F \cup \{C[0]\}$; $G \leftarrow G \setminus \{C[0]\}$; **continue**
        $G_c \leftarrow (V_c = C, E_c = \emptyset)$
        for each unique pair $(e_A, e_B) \in \binom{C}{2}$ **do**
            $P \leftarrow$ PREDICTCONFLICTPROBABILITY($G, e_A, e_B, \mathbf{p}, \mathscr{M}_{\text{RF}}$)
            if $P \geq \tau$ **then**
                $E_c \leftarrow E_c \cup \{\{e_A, e_B\}\}$
        $I_{\text{safe}} \leftarrow$ MAXIMUMINDEPENDENTSET(G_c)
        if $I_{\text{safe}} = \emptyset$ **then**
            $I_{\text{safe}} \leftarrow \{C[0]\}$
        for each $e \in I_{\text{safe}}$ **do**
            $F \leftarrow F \cup \{e\}$; $G \leftarrow G \setminus \{e\}$
return F

3.6.1 Computational Complexity and Amortization

The per-iteration time complexity within an SCC $s_i = (V_{s_i}, E_{s_i})$ decomposes into three operational stages [25]

1. **Line Digraph and PageRank Stage:** Constructing $L(s_i)$ and executing five PageRank iterations requires:

$$T_{\text{line}}(s_i) = O\left(|E_{s_i}| + \sum_{v \in V_{s_i}} \left[d^+(v) \times d^-(v) \right] \right) \tag{32}$$

   Extracting the top-K scoring candidates using a min-heap operates in $O(|E_{s_i}| \log K)$ time [25]

2. **Feature Extraction and Inference Stage:** For each of the $\binom{K}{2} = \frac{K(K-1)}{2}$ candidate pairs, evaluating shortest paths requires bidirectional Breadth-First Searches (BFS) bounded by $O(|V_{s_i}| + |E_{s_i}|)$ [25]

3. **MIS Resolution Stage:** Executing the minimum-degree independent set heuristic on G_c takes $O(K^2)$ time [5, 25]

Because $K \leq 20$ is a small constant, $\binom{K}{2} \leq 190$. The machine learning inference and MIS resolution require under 2 milliseconds per iteration [25] Critically, if the independent set isolates $b = |I_{\text{safe}}| \geq 2$ mutually safe edges, the heavy line-digraph construction and PageRank power iterations are amortized over b simultaneous deletions [25] The total number of global line-graph regenerations drops from $|F|$ to approximately:

$$N_{\text{iterations}} \approx \sum_{s_i} \frac{|F(s_i)|}{\bar{b}_{s_i}} \ll |F| \tag{33}$$

where $\bar{b}_{s_i}$ is the average safe batch size per iteration [25] This structural amortization directly circumvents the super-linear recomputation bottleneck of the baseline algorithm, providing dramatic speedups while maintaining competitive FAS cardinality [12, 25]

4 Experiments and Discussion

5 Conclusions

Acknowledgments

First and foremost, I would like to thank my supervisors, Dr. Panagiotis Lionakis and Dr. Michalis Mountantonakis, for their continuous guidance, valuable insights, and encouragement throughout this project.

A heartfelt thank you goes to my girlfriend, Eri, and my family—Ieronymos, Athina, Dr. Chrysanthi, and Dr. Aikaterini—for their love, patience, and for standing by me every step of the way.

I am also truly grateful to my friends—Andreas, Antonis, Dimitris, Filippos, Fotis, Giannis, Kostas, Mimis, Panagiotis, and Pavlos—for the memorable moments, constant support, and for keeping spirits high. Finally, thanks to my coworkers—Alexis, Dimitris, Nikos, Nikos, Spyros, Spyros, and Panayis—for their great collaboration, and positive energy.

References

[1] N. Ailon, M. Charikar, and A. Newman, Aggregating inconsistent information: ranking and clustering, *Journal of the ACM (JACM)*, 55(5):1–27, 2008. `https://doi.org/10.1145/1411509.1411513`.

[2] A. Baharev, H. Schichl, A. Neumaier, and T. Achterberg, An exact method for the minimum feedback arc set problem, *ACM Journal of Experimental Algorithmics*, 26:1–28, 2021. https://doi.org/10.1145/3446429.

[3] B. Berger and P. W. Shor, Approximation algorithms for the maximum acyclic subgraph problem, in *Proceedings of the 1st Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 236–243, 1990.

[4] P. Boldi, M. Rosa, M. Santini, and S. Vigna, Layered label propagation: A multiresolution coordinate-free ordering for compressing social networks, in *Proceedings of the 20th International Conference on World Wide Web (WWW)*, pages 587–596, 2011. https://doi.org/10.1145/1963405.1963488.

[5] R. Boppana and M. M. Halldórsson, Approximating maximum independent sets by excluding subgraphs, *BIT Numerical Mathematics*, 32(2):180–196, 1992. https://doi.org/10.1007/BF01994876.

[6] F. J. Brandenburg and K. Hanauer, Sorting heuristics for the feedback arc set problem, *Technical Report MIP-1104*, University of Passau, Germany, 2011.

[7] L. Breiman, Random forests, *Machine Learning*, 45(1):5–32, 2001. https://doi.org/10.1023/A:1010933404324.

[8] S. Brin and L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems*, 30(1–7):107–117, 1998. https://doi.org/10.1016/S0169-7552(98)00110-X.

[9] C. Budak, D. Agrawal, and A. El Abbadi, Limiting the spread of misinformation in social networks, in *Proceedings of the 20th International Conference on World Wide Web (WWW)*, pages 665–674, 2011. https://doi.org/10.1145/1963405.1963499.

[10] G. Di Battista, P. Eades, R. Tamassia, and I. G. Tollis, *Graph Drawing: Algorithms for the Visualization of Graphs*, Prentice-Hall, Upper Saddle River, NJ, 1999.

[11] P. Eades, X. Lin, and W. F. Smyth, A fast and effective heuristic for the feedback arc set problem, *Information Processing Letters*, 47(6):319–323, 1993. https://doi.org/10.1016/0020-0190(93)90079-O.

[12] V. Geladaris, P. Lionakis, and I. G. Tollis, Effective computation of a feedback arc set using PageRank, *Journal of Graph Algorithms and Applications*, 27(8):737–757, 2023. https://doi.org/10.7155/jgaa.00641.

[13] X. He, G. Song, W. Chen, and Q. Jiang, Influence blocking maximization in social networks under the competitive linear threshold model, in *Proceedings of the 12th SIAM International Conference on Data Mining (SDM)*, pages 463–474, 2012. https://doi.org/10.1137/1.9781611972825.40.

[14] D. S. Johnson, The NP-completeness column: An ongoing guide, *Journal of Algorithms*, 3(4):381–395, 1982. https://doi.org/10.1016/0196-6774(82)90032-3.

[15] R. M. Karp, Reducibility among combinatorial problems, in *Complexity of Computer Computations*, pages 85–103, Plenum Press, New York, 1972. https://doi.org/10.1007/978-1-4684-2001-2_9.

[16] J. Leskovec and A. Krevl, SNAP Datasets: Stanford large network dataset collection, http://snap.stanford.edu/data, 2014.

[17] N. S. Nikolov and P. Healy, Hierarchical drawing algorithms, in *Handbook of Graph Drawing and Visualization*, R. Tamassia, ed., pages 409–453, CRC Press, Boca Raton, FL, 2014.

[18] G. Ortali and I. G. Tollis, A new framework for hierarchical drawings, *Journal of Graph Algorithms and Applications*, 23(3):553–578, 2019. https://doi.org/10.7155/jgaa.00502.

[19] L. Page, S. Brin, R. Motwani, and T. Winograd, The PageRank citation ranking: Bringing order to the web, *Technical Report 1999-66*, Stanford InfoLab, Stanford University, 1999.

[20] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, Scikit-learn: Machine learning in Python, *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[21] S. Pemmaraju and S. Skiena, *Computational Discrete Mathematics: Combinatorics and Graph Theory with Mathematica*, Cambridge University Press, 2003.

[22] M. Simpson, V. Srinivasan, and A. Thomo, Efficient computation of feedback arc set at web-scale, *Proceedings of the VLDB Endowment*, 10(3):133–144, 2016. https://doi.org/10.14778/3021924.3021930.

[23] K. Sugiyama, S. Tagawa, and M. Toda, Methods for visual understanding of hierarchical system structures, *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-11(2):109–125, 1981. https://doi.org/10.1109/TSMC.1981.4308636.

[24] R. Tarjan, Depth-first search and linear graph algorithms, *SIAM Journal on Computing*, 1(2):146–160, 1972. https://doi.org/10.1137/0201010.

[25] G. Papadakis, ML-Batching-PageRankFAS: Machine Learning-Enhanced Feedback Arc Set Batching, *GitHub Repository*, 2026. https://github.com/papadogiorgis/ml-pagerankfas/.